Lab 2：Bomb Lab 实验报告

一、程序运行结果

本实验完成了 Phase 1—6 与 Honor 的求解函数。六个常规关卡的公开测试通过 60 项，Honor 的 10 个公开测试均通过。此外，使用公开列表之外的 seed 314159 进行完整验证，六关与 Honor 均通过，程序退出码为 0。

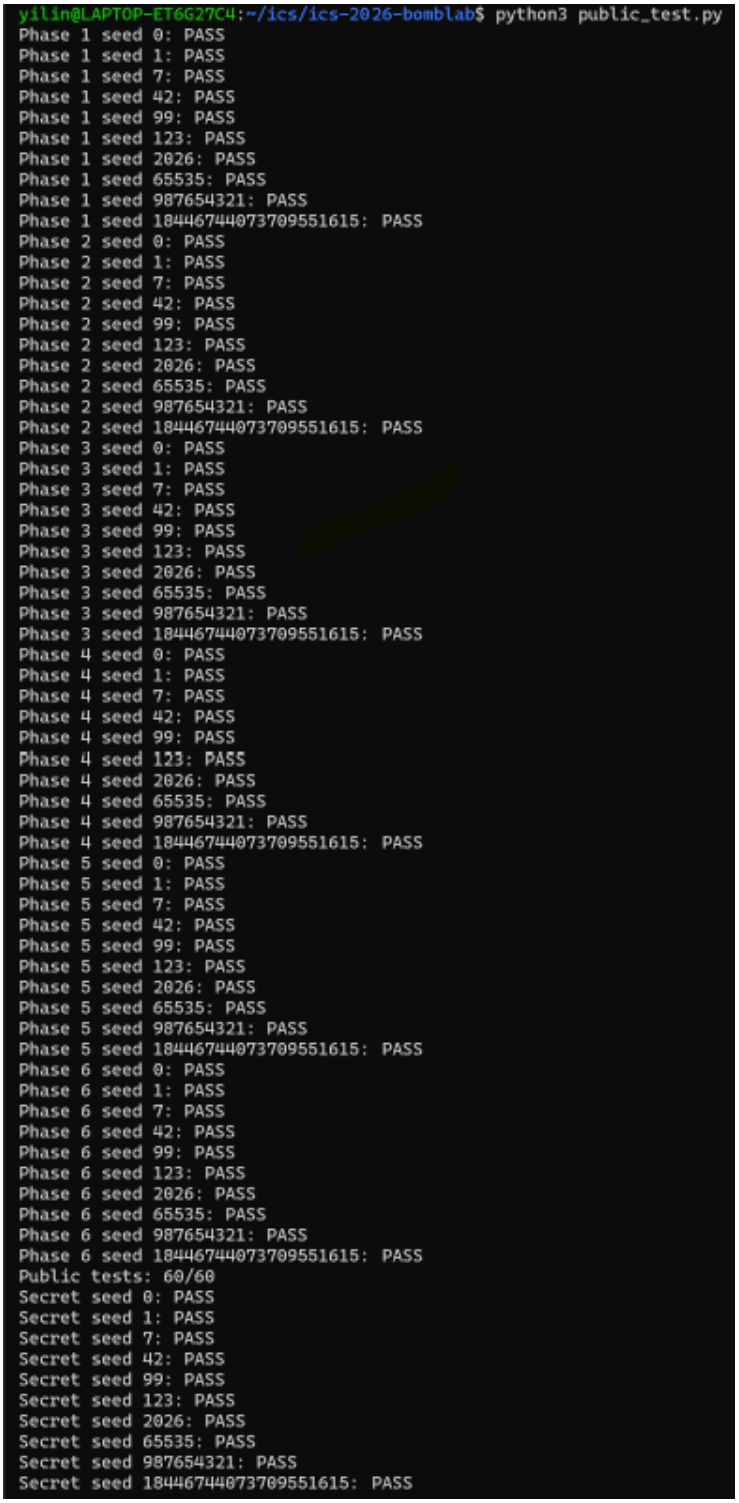


图 1 六个常规关卡与 Honor 的公开测试结果

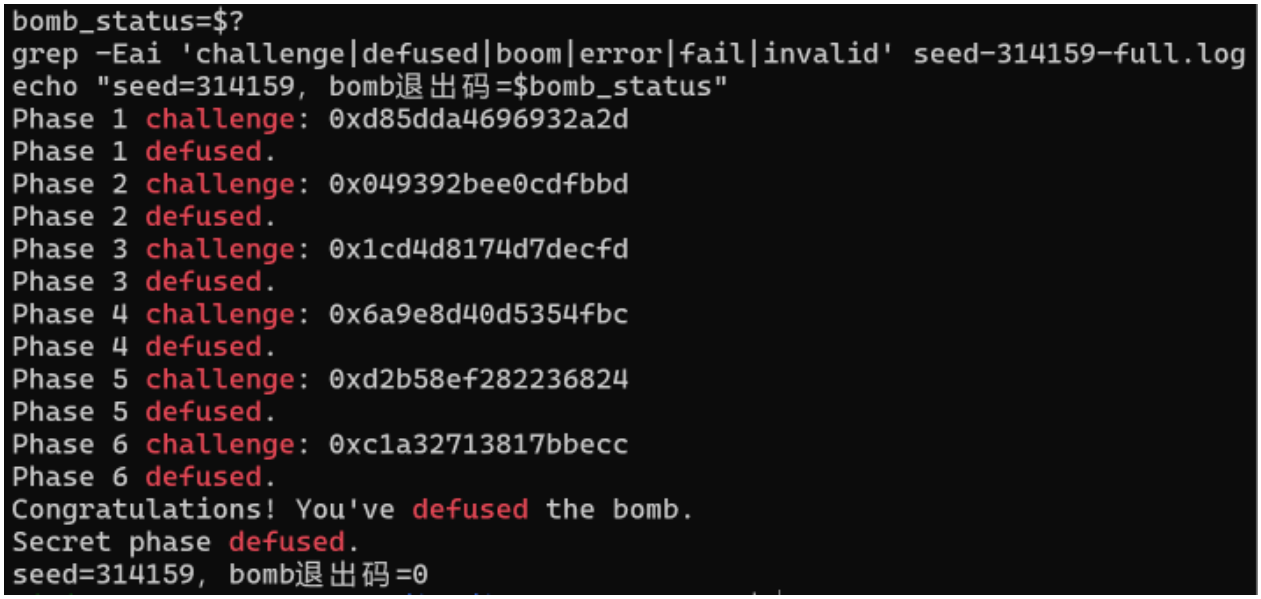


图 2 非公开 seed 314159 下六个常规关卡与 Honor 的验证结果

二、关卡思路流程

整体方法

使用 objdump 查看关卡函数、辅助函数与静态数据，结合调试结果恢复校验过程。首先确定输入字段数量，再追踪 challenge 的取位、运算、分支和内存访问，最后在 defuser.c 中实现通用求解规则。

各关主要使用 32 位整数运算。实现中采用 uint32_t 保存中间结果，保持溢出回绕、逻辑右移与循环移位的行为。以下汇编仅摘录决定性指令，地址均来自 bomb。

Phase 1：函数调用与返回值组合

本关要求输入一个无符号整数。phase_01 调用 p1_compute，随后将返回值与输入比较：

```
4020db: call   402050 <p1_compute>
4020e0: cmp    %eax,-0x14(%rbp)
4020e3: sete   %al
```

继续追踪可见，p1_compute 依次调用 p1_step2、p1_step0 和 p1_step1。其中，p1_step0 的关键运算为：

```
402024: lea    (%rdi,%rdi,4),%eax
402027: add    $0xa9a9,%esi
40202d: xor    %esi,%eax
```

这里的 lea 用于计算第一个参数的五倍。结合调用前的参数传递，可以恢复完整过程：先将 challenge 的低 32 位加 0x50ef，与高 32 位异或；再将低 32 位的五倍，与“右移 16 位后的低 32 位加 0xa9a9”异或；最后把第二个结果加上第一个结果的两倍，再异或 0x7e6b，得到口令。

求解函数按调用顺序组合中间结果，输出一个无符号十进制整数。

Phase 2：数组与循环递推

本关要求输入六个整数。首项由 challenge 右移 9 位后的低六位与 0x13 异或得到。其余五项在循环中逐项校验，关键指令为：

```
40217a: shl    $0x2,%eax
40217d: shr    %cl,%rdx
402180: and    $0xf,%edx
402183: xor    %edx,%eax
402185: lea    -0x6(%rcx,%rax,1),%edx
402189: mov    0x4(%rsi),%eax
40218c: cmp    %edx,%eax
```

shl $0x2 将前一项乘以 4；右移和掩码从 challenge 中提取四位数据，再与乘法结果异或。ecx 从 8 开始，每轮增加 2，因此最后加上的修正量依次为 2、4、6、8、10。随后指针前移 4 字节，检查下一个数组元素。

求解函数先计算首项，再按“乘 4、异或提取值、加修正量”的顺序生成后续五项。

Phase 3：分支与跳转表

本关输入分支编号及对应结果。第一项必须等于 challenge 的低三位；第二项由高 32 位与右移 3 位后的低 32 位异或后，送入分支函数计算。

p3_dispatch 的核心是跳转表：

```
402268: cmp    $0x7,%eax
40226b: ja     4022b5 <p3_dispatch+0x55>
40226d: notrack jmp *0x405820(,%rax,8)
```

索引乘以 8 后访问表项，说明每项保存一个 64 位目标地址。结合表中地址与后续跳转，编号 0—7 实际对应的函数依次为：

p3_case_5、p3_case_0、p3_case_1、p3_case_6、
p3_case_7、p3_case_2、p3_case_4、p3_case_3

因此，分支编号不能直接按函数名称中的数字对应。例如编号 0 进入 p3_case_5，其中：

```
402224: lea    0x685b(%rdi,%rdi,4),%eax
```

表示将公共参数乘以 5，再加 0x685b。其余分支同样逐项恢复，求解函数用 switch 保存实际映射，输出编号与计算结果。

Phase 4：递归搜索与回溯计算

本关以 challenge 的低七位作为目标，在 [0,127] 中进行二分式递归搜索。密钥由 challenge 右移 7 位后的低 32 位与高 32 位异或得到。

p4_walk 先计算中点并比较目标；相等时，将中点加 0x98 后与密钥异或，作为递归终止值。若不相等，则缩小区间并继续调用自身。右分支返回后的关键指令为：

```
402367: call   402340 <p4_walk>
40236c: lea    (%rax,%rax,2),%eax
40236f: xor    %ebx,%eax
402376: xor    $0x19,%eax
```

返回值先乘以 3，再与当前中点加密钥的结果异或，最后异或右分支常量 0x19；左分支采用相同结构，但常量为 0x14。

因此，最终口令由整条搜索路径及回溯过程共同决定。求解函数保留递归结构，在找到目标后逐层组合返回值。

Phase 5：结构体与链表遍历

本关要求输出两个整数。起点由 challenge 的低三位通过入口表选择，后继移动次数由随后两位确定。节点访问中的关键指令为：

```
402478: mov    (%rdx),%rdx
40247b: add    $0x1,%ecx
40247e: mov    0x8(%rdx),%esi
402481: add    0xc(%rdx),%eax
402484: lea    (%rax,%rax,4),%eax
402487: mov    %esi,(%r8)
40248a: xor    %esi,%eax
```

mov (%rdx),%rdx 表明节点起始位置保存后继指针；偏移 8 和 12 处分别读取 32 位数据。每访问一个节点，先将偏移 12 处的数据加到累加器中，再乘以 5，最后与偏移 8 处的数据异或。同时，程序保存当前节点偏移 8 处的字段，供第二项校验使用。

起点在进入后继循环前已经处理一次，因此移动后继指针 steps 次，共处理 steps + 1 个节点。求解函数用后继数组和字段数组模拟原链表，最终输出累加器及末节点偏移 8 处的字段值。

Phase 6：函数指针表与间接调用

本关根据 challenge 选择两次函数调用。低三位确定第一次表项，随后三位确定第二次表项。初始参数由 challenge 右移 6 位后的低 32 位与高 32 位异或得到。

关键调用过程为：

```
4025c6: call   *0x405e00(,%rax,8)
4025cd: xor    $0xfa82,%eax
4025d2: mov    %eax,%edi
4025d4: call   *0x405e00(%rbx)
4025df: add    $0xef67,%eax
```

第一次返回值先异或 0xfa82，再作为第二次调用的参数，第二次返回后加 0xef67，得到口令。第二次访问中，ebx 已保存按 0x38 掩码得到的字节偏移，相当于取出索引后乘以 8。

读取函数指针表可知，八个表项依次指向：

p6_op_3、p6_op_2、p6_op_5、p6_op_4、
p6_op_0、p6_op_0、p6_op_1、p6_op_5

各函数包含乘加、异或及循环移位，其中 rol $0x7,%eax 和 ror $0x2,%eax 分别表示 32 位循环左移 7 位、循环右移 2 位。求解函数按照表项映射实现各项运算，并保留两次调用的组合顺序。

Honor：入口布局与上下文校验

Honor 需要同时恢复入口文本布局和三个整数的校验规则。secret_gate 将输入复制到从 rbp-0xd0 开始的区域，随后执行：

```
4027b6: call   402730 <secret_copy>
4027bb: cmpl   $0x4e45504f,-0xb0(%rbp)
4027c5: jne    40279b <secret_gate+0x4b>
4027cd: lea    -0xac(%rbp),%rdi
4027d4: call   402670 <secret_phase>
```

检查位置与复制起点相差 0x20，即 32 字节。按小端序，0x4e45504f 对应字符 OPEN；其后地址被传给 secret_phase 解析三个整数。因此，解法使用 32 个 A 填充，紧接 OPEN 和三个整数。

数值校验同时依赖 challenge 与前六关形成的 context。第一个整数必须等于 context 的低两位；第二个整数结合两者高低 32 位的异或结果、context 的部分位及 secret_mix 返回值计算。secret_mix 的核心指令为：

```
402656: xor    $0xb047,%edi
40265c: shr    $0x3,%eax
40265f: xor    $0xf1f8,%eax
402664: add    %edi,%eax
```

即将同一参数分别进行异或、右移处理后相加。第三个整数再以第二个整数为基础，与密钥、第一个整数、常量及移位结果的和异或得到。

求解函数按上述布局构造输入，并使用同一 seed 下的六关口令生成对应上下文。测试时额外使用 --secret 输出 Honor 行，与前六行一起送入炸弹。公开测试及 seed 314159 的完整验证均通过。

三、参考资料与辅助工具说明

参考本届 Bomb Lab 实验文档、课程机器级程序表示相关课件及往届学长学姐实验报告。往届报告用于参考组织方式，各关计算规则依据本人的炸弹程序、解法及测试结果核对。

使用 objdump 查看反汇编与静态数据，结合 GDB 观察程序状态。ChatGPT 参与思路梳理、概念核对、常见错误提示、报告语言润色。不用于直接生成可提交代码；最终代码与最终输入/调试过程由本人独立实现与调试。